

low-rank matrix. In the middle regime of β , we need the sparse part to cover the intra-cluster attention and the low-rank part to approximate the inter-cluster attention.

We formalize this intuition in Theorem 1 (in bounds below we think of ϵ as a constant). All the proofs are in Appendix D.

Theorem 1. *Let M_β be the attention matrix in Process 1. Fix $\epsilon \in (0, 1)$. Let $R \in \mathbb{R}^{n \times n}$ be a matrix. Consider low-rank, sparse, and sparse + low-rank approximations to M_β .*

1. **High temperature:** Assume $\beta = o(\log n / \log d)$.

(a) **Low-rank:** *There exists R with $n^{o(1)}$ rank (and hence $n^{1+o(1)}$ parameters) such that $\|M_\beta - R\|_F \leq \epsilon n$.*

(b) **Sparse:** *If R has sparsity $o(n^2)$, then $\|M_\beta - R\|_F \geq \Omega(n)$.*

2. **Mid temperature:** Assume $(1 - \Delta^2) \log n \leq \beta \leq O(\log n)$.

(a) **Sparse + low-rank:** *There exists a sparse + low-rank R with $n^{1+o(1)}$ parameters with $\|M_\beta - R\|_F \leq \epsilon n$.*

(b) **Low-rank:** *If R is such that $n - \text{rank}(R) = \Omega(n)$, then $\|M_\beta - R\|_F \geq \Omega(n)$.*

(c) **Sparse:** *If R has sparsity $o(n^2)$, then $\|M_\beta - R\|_F \geq \Omega(n)$.*

3. **Low temperature:** Assume $\beta = \Omega(\log n)$.

(a) **Low-rank:** *If R is such that $n - \text{rank}(R) = \Omega(n)$, then $\|M_\beta - R\|_F \geq \Omega(e^{\beta(1-\Delta^2)})$.*

(b) **Sparse:** *There exists R with sparsity $O(n)$ such that $\|M_\beta - R\|_F \leq \epsilon \cdot e^{\beta(1-\Delta^2)}$*

4 Scatterbrain: Unifying Sparse and Low-rank Attention

We present Scatterbrain, and show that it approximates attention accurately and efficiently. Section 4.1 describes the challenges of designing an accurate and efficient approximation, and how obvious baselines such as Robust PCA or a simple combination of sparse attention and low-rank attention fail to meet both criteria. Section 4.2 demonstrates how Scatterbrain address the challenges (Fig. 1 contains a schematic of Scatterbrain). In Section 4.3, we show that Scatterbrain is unbiased with provably lower variance than low-rank baselines such as Performer.

Fig. 3 shows a qualitative comparison between different methods of approximating the attention matrix: Robust PCA is accurate but slow, sparse (e.g., Reformer), and low-rank (e.g., Performer) attention are fast and memory-efficient but may not be very accurate, while Scatterbrain is more accurate than its sparse and low-rank counterparts while remaining just as efficient.

More details about the efficient implementation of Scatterbrain are in Appendix C.

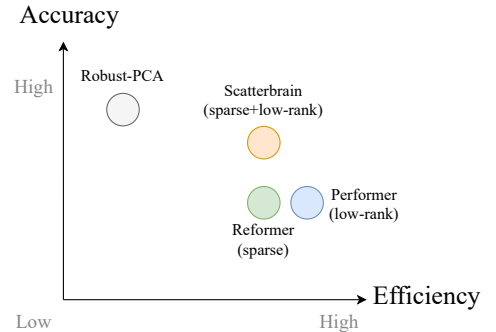


Figure 3: Qualitative comparison of approx. accuracy and efficiency, among Robust PCA, sparse (Reformer) and low-rank (Performer) attention, and Scatterbrain. Scatterbrain is more accurate while being efficient.

4.1 Challenges of Designing an Accurate and Efficient Sparse + Low-rank Approximation

We seek a sparse + low-rank approximation of the attention matrix² A that is *both* accurate and efficient. The natural theoretical baseline of Robust PCA is too slow and requires too much memory, while the most straightforward way of combining sparse attention and low-rank attention fails due to double counting on the support of the sparse attention.

²For simplicity of discussion, we consider the unnormalized attention matrix $A = \exp(QK^\top)$, omitting the usual scaling of $\sqrt{d}$ and the softmax normalization constant.

1. If the goal is accuracy, Robust PCA is the most studied algorithm to find a sparse + low-rank approximation to a given matrix. It relaxes the NP-hard problem of finding the best sparse + low-rank approximation into a convex optimization problem, with the nuclear norm and ℓ_1 constraints. Even though it can be solved in polynomial time, it is orders of magnitude too slow to be used in each iteration of a training loop. Moreover, it requires materializing the attention matrix, which defeats the main purpose of reducing compute and memory requirements.
2. On the other hand, one efficient way to get sparse + low-rank approximation of an attention matrix is to simply add the entries of a sparse approximation S (say, from Reformer) and a low-rank approximation $\tilde{Q}\tilde{K}^\top$ for $\tilde{Q}, \tilde{K} \in \mathbb{R}^{n \times m}$ (say, from Performer). The sparse matrix S typically has support determined randomly [16], by LSH [22, 36], or by clustering [53]. On the support of S , which likely includes the locations of the large entries of the attention matrix A , the entries of S match those of A . One can multiply $(S + \tilde{Q}\tilde{K}^\top)V = SV + \tilde{Q}(\tilde{K}^\top V)$ efficiently because S is sparse, and grouping $\tilde{Q}(\tilde{K}^\top V)$ reduces the matrix multiplication complexity when $m \ll n$, from $O(n^2m)$ to $O(nmd)$. The approximation $S + \tilde{Q}\tilde{K}^\top$ matches $\tilde{Q}\tilde{K}^\top$ outside $\text{supp}(S)$, hence it could be accurate there if $\tilde{Q}\tilde{K}^\top$ is accurate. However, $S + \tilde{Q}\tilde{K}^\top$ will not be accurate on the support of S due to the contributions from both S and from $\tilde{Q}\tilde{K}^\top$. Adjusting $\tilde{Q}\tilde{K}^\top$ to discount the contribution from S is difficult, especially if we want to avoid materializing $\tilde{Q}\tilde{K}^\top$ for efficiency.

4.2 Scatterbrain: Algorithm Intuition and Description

The simple insight behind our method is that on the support of the sparse matrix S , instead of trying to match the entries of the attention matrix A , we can set the entries of S to discount the contribution from the low-rank part $\tilde{Q}\tilde{K}^\top$. This way, the approximation $S + \tilde{Q}\tilde{K}^\top$ will match A exactly on the support of S , and will match $\tilde{Q}\tilde{K}^\top$ outside $\text{supp}(S)$, which means it will still be accurate there if $\tilde{Q}\tilde{K}^\top$ is accurate. We do not need to materialize the full matrix $\tilde{Q}\tilde{K}^\top$ as need a subset of its entries is required, hence our approximation will be compute and memory efficient.

Scatterbrain thus proceeds in three steps: we construct a low-rank approximation $\tilde{Q}\tilde{K}^\top \approx A$, and construct a sparse matrix S such that $S + \tilde{Q}\tilde{K}^\top$ matches A on the support of S , then finally multiply SV and $\tilde{Q}(\tilde{K}^\top V)$ and combine the result. More specifically:

1. **Low-rank Approximation.** We define a procedure LOWRANK that returns two matrices $\tilde{Q}, \tilde{K} \in \mathbb{R}^{n \times m}$ such that $\tilde{Q}\tilde{K}^\top$ approximates A . In particular, we use a randomized kernel feature map $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^m$ where $\phi(x) = \frac{1}{\sqrt{m}} \exp(Wx - \|x\|^2/2)$ with $W \in \mathbb{R}^{m \times d}$ randomly sampled, entry-wise, from the standard normal distribution $\mathcal{N}(0, 1)$. We apply ϕ to each row vector of Q, K matrices, and denote $\tilde{Q} = \phi(Q)$ and $\tilde{K} = \phi(K)$ (row-wise). Note that we do not materialize $\tilde{Q}\tilde{K}^\top$.
2. **Sparse Approximation.** We define a procedure SPARSE that returns a sparse matrix S that matches $A - \tilde{Q}\tilde{K}^\top$ on $\text{supp}(S)$. In particular, using a family of locality sensitive hash functions, compute the hash codes of each query and key vectors in Q, K matrices (row-wise). Let $\mathcal{S}$ be the set of locations (i, j) where q_i and k_j have the same hash codes (i.e, fall into the same hash bucket). Let S be the sparse matrix whose support is $\mathcal{S}$, and for each $(i, j) \in \mathcal{S}$, define

$$S_{i,j} = \exp(q_i^\top k_j) - \phi(q_i)^\top \phi(k_j) = \exp(q_i^\top k_j) - \tilde{q}_i^\top \tilde{k}_j, \quad (1)$$

where $q_i, k_j, \tilde{q}_i, \tilde{k}_j$ are the i -th and j -th rows of $Q, K, \tilde{Q}, \tilde{K}$ respectively. Note that we do not materialize $\tilde{Q}\tilde{K}^\top$.

3. **Scatterbrain Approximation.** With $\tilde{Q}, \tilde{K}$ returned from LOWRANK and S from SPARSE, we compute the (unnormalized) attention output with

$$\tilde{O} = (\tilde{Q}\tilde{K}^\top + S)V = \tilde{Q}(\tilde{K}^\top V) + SV. \quad (2)$$

The precise algorithm, including the normalization step, as well as the causal/unidirectional variant, is described in Appendix C. We also note Scatterbrain’s flexibility: it can use different kinds of low-rank and sparse approximation as its sub-components. The combination of Reformer and Performer is simply one instance of Scatterbrain. Instead of using Reformer as a sparse component, we could use local attention [5] or random block-sparse attention [16]. Instead of using Performer [17] as a low-rank component, we could also use Linear attention [35] or global tokens as in BigBird [70].

The Scatterbrain method would work exactly the same way. As long as the low-rank component is unbiased (e.g., Performer), its combination with any sparse component in Scatterbrain would yield an unbiased estimator of the attention matrix as shown below.

4.3 Scatterbrain: Analysis

Our method combines a low-rank approximation $\tilde{Q}\tilde{K}^\top$ (which has rank $m \ll n$) with a sparse approximation S . We argue that it is accurate (lower approximation error than baselines) and efficient (scaling the same as sparse or low-rank alone). The main insight of the analysis is that our approximation is exact for entries on the support of S (picked by LSH), which are likely to be large. For entries not in the support of S (likely to be small), our approximation matches the low-rank part (Performer) $\tilde{Q}\tilde{K}^\top$, which is unbiased and has low variance for these entries. As a result, Scatterbrain retains the unbiasedness of Performer [17] but with strictly lower variance.

We compare Scatterbrain to its low-rank baseline (Performer) and sparse baseline (Reformer). Performer is also based on the kernel approximation ϕ , and simply uses $\tilde{Q}\tilde{K}^\top$ to approximate the attention matrix A . Reformer uses LSH to identify large entries of A , then compute a sparse matrix S such that $S_{ij} = \exp(q_i^\top k_j)$ for $ij \in \text{supp}(S)$.

Accuracy: Because of the way S is defined in Eq. (1), $\tilde{Q}\tilde{K}^\top + S$ matches $A = \exp(QK^\top)$ exactly on locations $(i, j) \in \mathcal{S}$, which are locations with likely large values. This addresses a weakness of low-rank methods (e.g., Performer) where the low-rank estimate is not accurate for locations with large values. We analyze the expectation and variance per entry of our estimator below (proof in Appendix D).

Theorem 2. Define $\sigma(q, k) = \exp(q^\top k)$, $\hat{\sigma}^{\text{pfe}}$ as Performer’s estimator and $\hat{\sigma}^{\text{sbe}}$ as Scatterbrain estimator. Denote $\mathcal{S}^{d-1} \subset \mathbb{R}^d$ as the unit sphere. Suppose $q, k \in \mathcal{S}^{d-1}$ are such that $\|q - k\| < \tau$. Then:

$$\mathbb{E}[\hat{\sigma}^{\text{sbe}}(q, k)] = \sigma(q, k), \quad \text{Var}[\hat{\sigma}^{\text{sbe}}(q, k)] = (1 - p) \cdot \text{Var}[\hat{\sigma}^{\text{pfe}}(q, k)] < \text{Var}[\hat{\sigma}^{\text{pfe}}(q, k)] \quad (3)$$

where $p = \exp(-\frac{\tau^2}{4-\tau^2} \ln d - O_\tau(\ln \ln d))$.

Hence Scatterbrain is unbiased, similar to Performer [17], but with strictly lower variance. The variance is small if $\exp(q^\top k)$ is small (since $\text{Var}(\hat{\sigma}^{\text{pfe}}(q, k))$ will be small), or if $\exp(q^\top k)$ is large (since the probability of not being selected by LSH, $1 - p$, will be small). In Fig. 4, we plot the per-entry MSE of different methods from Theorem 2 when approximating the unnormalized softmax attention $\exp(QK^\top)$. Scatterbrain can approximate well both small entries (similar to the low-rank baseline, Performer), as well as large entries (similar to the sparse baseline, Reformer). Thus Scatterbrain has much lower MSE than Performer for large entries, and lower MSE than Reformer for small entries.

Efficiency: In Eq. (2), the computation SV is efficient because S is sparse, and $\tilde{Q}(\tilde{K}^\top V)$ is efficient because of the way we associate matrix multiplication (scaling as $O(nmd)$ instead of $O(n^2d)$, which is much bigger if $m \ll n$).

We validate these two properties of our approach in Section 5.

5 Experiments

We validate three claims that suggest Scatterbrain provides an accurate and efficient approximation to attention matrices, allowing it to outperform its sparse and low-rank baselines on benchmark datasets.

- In Section 5.1, we evaluate the approximation error and testing accuracy of different approximation methods on pre-trained models such as BigGAN and Vision Transformer. We show that the approximation by Scatterbrain is close to the Robust PCA oracle and up to $2.1\times$ lower approximation error than other efficient baselines.

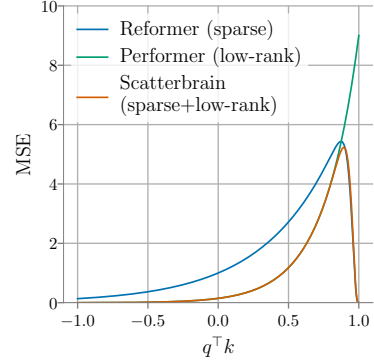


Figure 4: Per-entry MSE for different approximations, across a range of magnitude of $q^\top k$. Scatterbrain has low MSE for both small and large entries, thus outperforming its sparse (Reformer) and low-rank (Performer) counterparts.

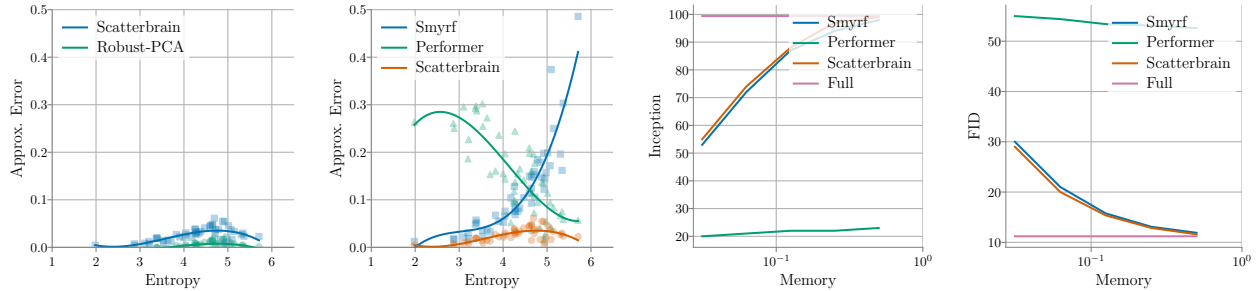


Figure 5: First: approximation comparison between Scatterbrain and its “lowerbound” Robust PCA. Second: comparison of error vs. entropy among SMYRF, Performer and Scatterbrain, three representatives of sparse, low-rank and sparse+low-rank approximations. Third and forth: Inception score (higher is better) and FID score (lower is better) of different attention variants for pretrained BigGAN.

- In Section 5.2, we validate that when trained end-to-end, Scatterbrain outperforms baselines (sparse or low-rank attention) on a wide variety of benchmark tasks, including language modeling, classification, and the Long-range Arena (LRA) benchmarks. Scatterbrain achieves up to 5 points higher average accuracy on the LRA benchmark compared to Performer and Reformer.
- In Section 5.3, we demonstrate the scalability of Scatterbrain, showing that it has comparable memory and time usage with simpler baselines (sparse or low-rank alone) across a range of input sequence lengths (Section 5.3), while requiring up to $12\times$ smaller memory than full attention.

All details (hyperparameters, data splits, etc.), along with additional experiments, are in Appendix E.

5.1 Scatterbrain’s Approximation Accuracy

We evaluate Scatterbrain’s approximation accuracy in three steps: (1) compare it with of Robust PCA (sparse+low-rank), our theoretical foundation and oracle (2) compare it with SMYRF³ [22], Performer [17], which are popular variants of sparse and low-rank approximation to attention respectively and a naive baseline that directly adds SMYRF and Performer, (3) evaluate the inference accuracy when replacing full attention with Scatterbrain approximation. Scatterbrain achieves error within 20% of the oracle robust PCA, and up to $2.1\times$ lower error than SMYRF and Performer. When serving as a drop-in replacement for full attention, even without training, Scatterbrain can reduce the attention memory of Vision Transformer by 98% at the cost of only 0.8% drop of accuracy.

Setup: We use the attention matrices from pre-trained BigGAN and T2T-ViT. BigGAN is a state-of-the-art model in Image Generation for ImageNet. BigGAN has a single attention layer at resolution 64×64 (4096 queries). T2T-ViT has 14 attention layers. Scatterbrain sets the ratio between SMYRF and Performer based on the entropy of an observed subset of attention matrices in different layers. We allocate more memory to the low-rank component compared to the sparse part if the entropy is high.

Scatterbrain and Robust PCA: We first show that Scatterbrain approximates pre-trained attention matrices $10^5\times$ faster while its approximation error is within 20% on average. We also provide an example visualization on 100 attention matrices from the BigGAN generation process in Figure 5 (left).

Scatterbrain vs. SMYRF and Performer: We show that Scatterbrain approximates pre-trained dense attention matrices with very low error compared to sparse (Reformer) or low-rank (Performer). Measuring Frobenius approx. error on the BigGAN image generation task, Scatterbrain achieves $2\times$ lower error compared to Performer.

Table 1: Top-1 Accuracy of pre-trained T2T Vision Transformer on ImageNet with different attention replacements. Error represents the average normalized approximation error to full attention.

Attention	Top-1 Acc	Error (avg)
Full Attention	81.7%	-
SMYRF	79.8%	11.4%
Performer	80.1%	7.5%
Baseline SMYRF + Performer	79.7%	12.6%
Scatterbrain	80.7%	5.3%

³SMYRF is a variant of Reformer that does not require the key and query to be the same, which is necessary for experiments in this section.